

PPS-2(LAB-6)

QUE-1: Family and Person classes.

Ans:- #include <cmath>

#include <cstdio>

#include <vector>

#include <iostream>

#include <algorithm>

using namespace std;

class Family;

class Person {

 string name;

 int age;

public:

 Person(string name = " ", int age = 0) {

 this-> name = name;

 this -> age = age;

 }

 friend class Family;

};

class Family {

 Person p[20];

public:

 void add(int n) {

```

for(int i=0; i<n; i++) {
    string name;
    int age;
    // getline(cin>>ws, name);
    cin >> name;
    cin >> age;
    p[i] = Person(name, age);
}
}

void print(int n) {
    for(int i=0; i<n; i++) {
        cout << p[i].age<< " " << p[i].name << endl;
    }
}

};

int main() {
    int n;
    cin >> n;
    Family F;
    F.add(n);
    F.print(n);

    return 0;
}

```

QUE-2: Product class - constructor and copy constructor

Ans:- `#include <cmath>`

`#include <cstdio>`

`#include <vector>`

`#include <iostream>`

`#include <iomanip>`

`#include <algorithm>`

`using namespace std;`

`class Product {`

`string nameOfProduct;`

`double Item_price;`

`public:`

`Product(string nameOfProduct, double Item_price) {`

`this-> nameOfProduct = nameOfProduct;`

`this-> Item_price = Item_price;`

`}`

`Product(const Product& c){`

`nameOfProduct = c.nameOfProduct;`

`Item_price = (c.Item_price * 2);`

`}`

`void print() {`

`cout << nameOfProduct << " " << fixed << setprecision(2) << Item_price << endl;`

```

    }
};

int main() {
    /* Enter your code here. Read input from STDIN. Print output to STDOUT */
    string name;
    double price;
    cin >> name >> price;
    Product p(name, price);
    Product p1 = Product(p);
    p1.print();

    return 0;
}

```

QUE-3: A shopping cart.

Ans:- #include<iostream>

#include<string>

using std::cout;

using std::cin;

using std::endl;

using std::string;

class ShoppingCart;

```
class Item{
public:
    string name;
    int price;
    Item(string name=" ", int price=0){
        this->name=name;
        this->price=price;
    }
};
```

```
class ShoppingCart{
    Item i[20];
    int count[20]={};
    int l=0;

public:
    void add_item(Item j, int quantity){
        for(int k=0; k<20; k++){
            if(j.name==i[k].name){
                count[k]+=quantity;
                return;
            }
        }
        i[l].name=j.name;
        count[l]=quantity;
        i[l].price=j.price;
```

```

        l++;
    }

    int calculate_bill(){
        int totalp=0;
        for(int k=0;k<l;k++){
            totalp+=i[k].price*count[k];
        }
        return totalp;
    }

    void print(){
        for(int k=0; k<l; k++){
            cout << i[k].name << " " << count[k] << endl;
        }
    }
};

int main() {

    Item items[10];

    int input_items_count, price;
    string name;
    cin >> input_items_count;
    for(int i = 0; i < input_items_count; i++) {
        getchar();
        cin >> name >> price;
        items[i] = Item(name, price);
    }
}

```

```
}
```

```
ShoppingCart sc;  
int purchase_entries, quantity;  
cin >> purchase_entries;  
for(int i = 0; i < purchase_entries; i++) {  
    getchar();  
    cin >> name >> quantity;  
    int j;  
    for(j = 0; j < input_items_count; j++) {  
        if(items[j].name == name)  
            break;  
    }  
    sc.add_item(items[j], quantity);  
}  
cout << sc.calculate_bill() << endl;  
sc.print();  
  
return 0;  
}
```

QUE-4: A Bus on a Route: profit or loss?

Ans:- #include<iostream>

#include<string>

#include<iomanip>

```
using std::string;
```

```
using std::cin;
```

```
using std::cout;
```

```
class Route;
```

```
class Bus {
```

```
private:
```

```
    static double petrol_price_per_litre;
```

```
    static double diesel_price_per_litre;
```

```
    string bus_id, fuel_type;
```

```
    double mileage_per_litre;
```

```
    int max_passengers;
```

```
public:
```

```
    Bus(string bus_id, string fuel_type, double mileage_per_litre, int  
max_passengers) {
```

```
        this->bus_id = bus_id;
```

```
        this->fuel_type = fuel_type;
```

```
        this->mileage_per_litre = mileage_per_litre;
```

```
        this->max_passengers = max_passengers;
```

```
    }
```

```
    static void change_petrol_price(double petrolprice) {
```

```
        petrol_price_per_litre = petrolprice;
```

```
    }
```



```
static void change_diesel_price(double dieselprice) {  
    diesel_price_per_litre = dieselprice;  
}
```

```
friend double calculate_profit(const Route& r, const Bus& b);  
};
```

```
double Bus::petrol_price_per_litre = 80.88;  
double Bus::diesel_price_per_litre = 75.77;
```

```
class Route {
```

```
private:
```

```
    string route_id, source, destination;
```

```
    double distance;
```

```
    int fare_per_passenger;
```

```
public:
```

```
    Route(string route_id, string source, string destination, double distance, int  
    fare_per_passenger) {
```

```
        this->route_id = route_id;
```

```
        this->source = source;
```

```
        this->destination = destination;
```

```
        this->distance = distance;
```

```
        this->fare_per_passenger = fare_per_passenger;
```

```
    }
```

```

    friend double calculate_profit(const Route& r, const Bus& b);
};

double calculate_profit(const Route& r, const Bus& b) {
    if (b.fuel_type != "petrol" && b.fuel_type != "diesel") {
        return 0.0; // Invalid fuel type
    }

    double fuel_price;
    if (b.fuel_type == "petrol")
        fuel_price = Bus::petrol_price_per_litre;
    else
        fuel_price = Bus::diesel_price_per_litre;

    return (r.fare_per_passenger * b.max_passengers) - ((r.distance /
b.mileage_per_litre) * fuel_price);
}

int main() {
    static double petrol_price_per_litre, diesel_price_per_litre;
    cin >> petrol_price_per_litre >> diesel_price_per_litre;
    Bus::change_petrol_price(petrol_price_per_litre);
    Bus::change_diesel_price(diesel_price_per_litre);

    string bus_id, fuel_type;
    double mileage_per_litre;
    int max_passengers;
    cin >> bus_id >> fuel_type >> mileage_per_litre >> max_passengers;

```

```

Bus bus(bus_id, fuel_type, mileage_per_litre, max_passengers);

getchar(); // Removing newline from input buffer
string route_id, source, destination;
double distance;

int fare_per_passenger;
cin>>route_id >> source >> destination >> distance >> fare_per_passenger;
Route route(route_id, source, destination, distance, fare_per_passenger);
cout << std::fixed << std::setprecision(2) << calculate_profit(route, bus);

return 0;
}

```

QUE-5: A small Company.

```

#include<iostream>
#include<string>
using namespace std;
using std::cin;
using std::cout;
using std::string;
using std::endl;

class Company;

```

```
class Employee{
    string employee_id, employee_name, designation;
    int salary;
public:
    Employee(string employee_id=" ", string employee_name=" ", string
designation=" ", int salary=0){
        this->employee_id=employee_id;
        this->employee_name=employee_name;
        this->designation=designation;
        this->salary=salary;
    }
    void increment(int i){
        salary+=i;
    }
    friend class Company;
};
```

```
class Company{
    string name;
    Employee e[10];
    int l=0;
public:
    Company(string name){
        this->name=name;
    }
    void add_employee(Employee emp){
```

```

        for(int i=0; i<l; i++){
            if(e[i].employee_id==emp.employee_id)
                return;
        }
        e[l]=emp;
        l++;
    }
    void print(){
        for(int i=0; i<l; i++)
            cout << e[i].employee_id << " " << e[i].employee_name << " " <<
e[i].designation << " " << e[i].salary << endl;
    }
    void increase_salary(string employee_id, int increment_amount){
        for(int i=0; i<l; i++){
            if(e[i].employee_id==employee_id){
                e[i].increment(increment_amount);
            }
        }
    }
};

```

```

int main() {
    string company_name;
    cin >> company_name;
    Company cmp("MyCompany");

    int n;

```

```

cin >> n;

string employee_id, employee_name, designation;

int salary;

Employee emp;

for(int i = 0; i < n; i++) {
    getchar(); // Removing newline from input buffer

    cin >> employee_id >> employee_name >> designation >> salary;

    emp = Employee(employee_id, employee_name, designation, salary);

    cmp.add_employee(emp);
}


int m = 0, increment_amount;

cin >> m;

for(int i = 0; i < m; i++) {
    getchar(); // Removing newline from input buffer

    cin >> employee_id >> increment_amount;

    cmp.increase_salary(employee_id, increment_amount);
}

cmp.print();


return 0;
}

```